



GSI
Cassie Jeng

CS61C

Great Ideas
in
Computer Architecture
(a.k.a. Machine Structures)



Assistant
Teaching Professor
Lisa Yan

RISC-V Instruction Formats I



Machine Language

- Machine Language
- R-Type Instructions
- I-Type Instructions
- I*-Type Instructions, Load Instructions
- S-Type Instructions (Store)

Great Idea #1: Abstraction (Levels of Representation/Interpretation)

High Level Language Program (e.g., C)

```
temp = v[k];  
v[k] = v[k+1];  
v[k+1] = temp;
```

Compiler

Assembly Language Program (e.g., RISC-V)

```
lw      x3, 0(x10)
lw      x4, 4(x10)
sw      x4, 0(x10)
sw      x3, 4(x10)
```

Assembler

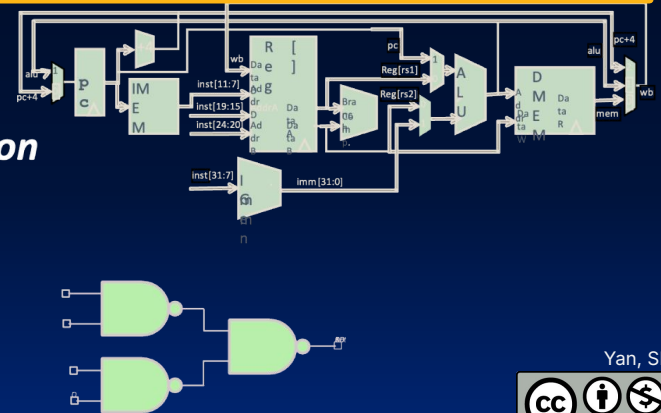
Machine Language Program (RISC-V)

1000	1101	1110	0010	0000	0000	0000	0000
1000	1110	0001	0000	0000	0000	0000	0100
1010	1110	0001	0010	0000	0000	0000	0000
1010	1101	1110	0010	0000	0000	0000	0100

Hardware Architecture Description (e.g., block diagrams)

Architecture Implementation

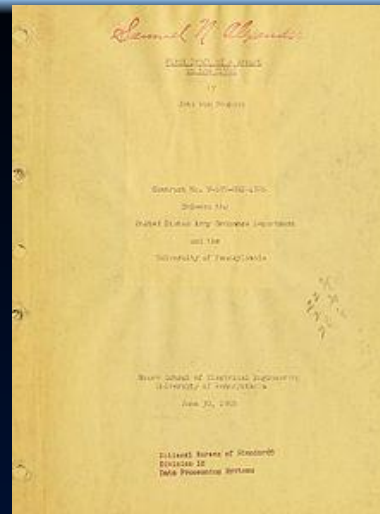
Logic Circuit Description (Circuit Schematic Diagrams)



History's Big Idea: Stored Program Computer

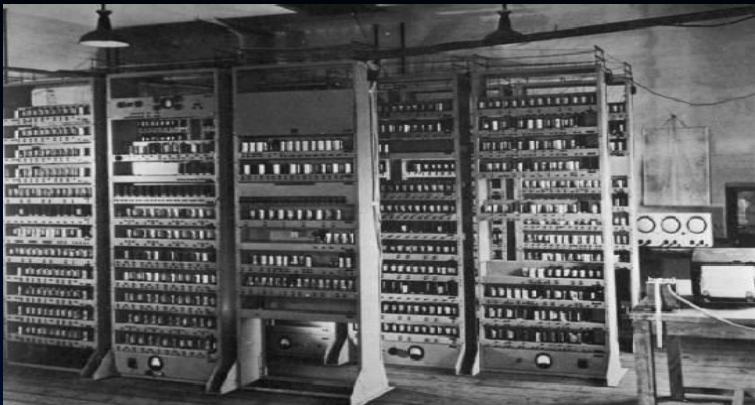
- **Stored-Program Computer** AKA the "von Neumann" computer is named after the widely distributed tech report.
 - Based on discussions of EDVAC (1944-): Eckert, Mauchly; anticipated earlier by Turing and Zuse
- Instructions are represented as bit patterns (i.e., numbers).
- Therefore, entire programs can be stored to memory to be read or written, just like data.
 - This means reprogramming can happen quickly (seconds); don't have to rewire computer (days).

CS61C core concepts reinforce the stored-program computer model!



First Draft of a Report on the EDVAC
By John von Neumann
Contract No. W-670-ORD-4926
Between the United States Army
Ordnance Department and the
University of Pennsylvania Moore
School of Electrical Engineering,
University of Pennsylvania
June 30, 1945

Quick CS History



- EDSAC: Cambridge, 1949:
First General **Stored-Program**
Electronic Computer
- 17-bit words (35-bit double word), binary 2's complement
 - Memory: 512 words
 - Multiplication: 6ms
 - Subroutines



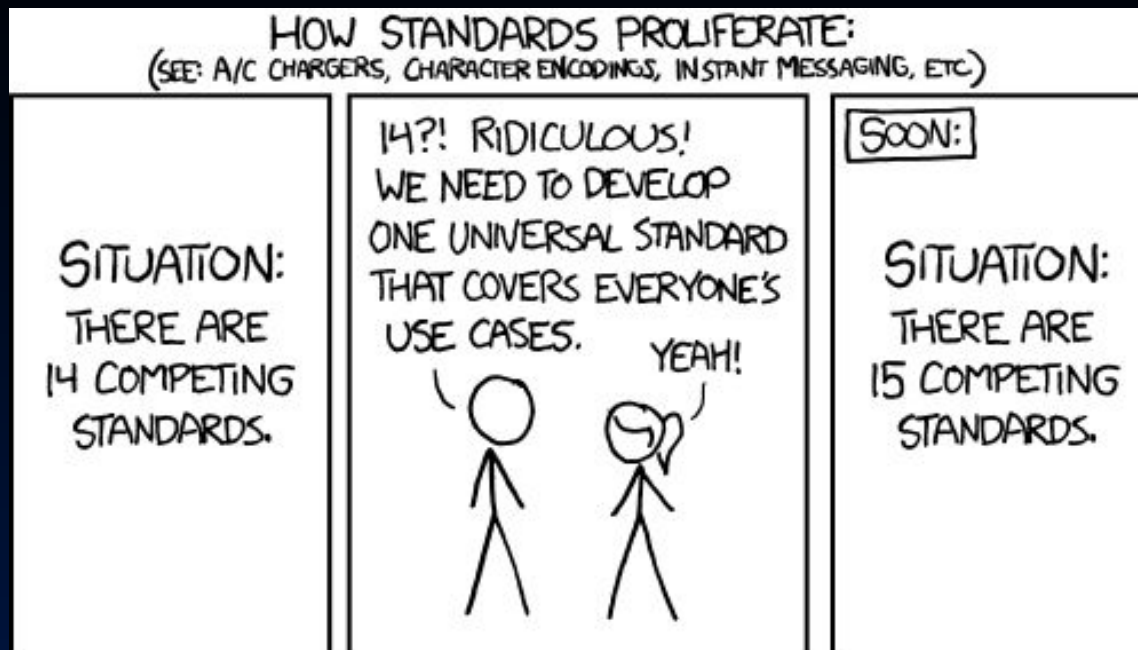
- IBM 701 (1952): IBM's first commercial scientific computer
- 36-bit sign-magnitude words
 - Memory: 2048 words with vacuum tubes
 - Multiplication: <0.5 ms
 - Two programmer-accessible registers: Accumulator, Multiplier/Quotient



Core Concepts Reinforce Stored Program Computer

- Everything (instructions, data words) has a memory address.
 - **Both branches and jumps use instruction memory addresses.**
 - The **Program Counter (PC)** register keeps **address of instruction** being executed.
 - PC is effectively a pointer to memory
 - Intel calls it Instruction Pointer (PC).
- Programs are distributed in *binary form*, i.e., as assembled machine code.
 - **Programs are each bound to a specific instruction set.**
 - Different instruction sets, e.g., for phone vs. PCs.
 - Many instruction sets are backwards-compatible and evolve over time, e.g., latest PCs with x86 ISA today can still run programs from Intel 8088 (1981)!

Today: Understand the why behind RISC-V ISA



Just one standard in this class:
RV32I ISA (RISC-V 32-bit Integer Instructions)
It's beautifully designed!

[xkcd](#), [explainxkcd](#)



[review]

- `add x18, x19, x10` → `00000000101010011000100100110011`
- assembly code machine code

- Remember: Computer only understands 1s and 0s.
Text like "**add x18,x19,x10**" is meaningless to hardware.

Let's learn instruction formats. Most of this lecture is on the **CS61C reference card**. No need to memorize exact machine code/bits!



RV32I Instructions: 32 bits wide, 6 Formats

- RISC-V simplicity: Split everything into **words (32-bit chunks)**.
 - Recall 32b registers; **lw**, **sw** access memory one word at a time.
 - Similarly, RISC-V **instructions are also 32-bit words**, fixed size
 - We cover RV32I (32-bit integer instructions).
 - Same 32-bit instructions used for RV32, RV64, RV128
- RISC-V simplicity: The ISA defines six basic **instruction formats**.
 - 32b instruction word divided into **fields**. Each field tells processor something about the instruction.
 - For similar instructions, use same format, same field definitions.
 - Today; R-, I-, S-Type Instruction Formats
 - Next time: B-, U-, J-Type Instruction Formats

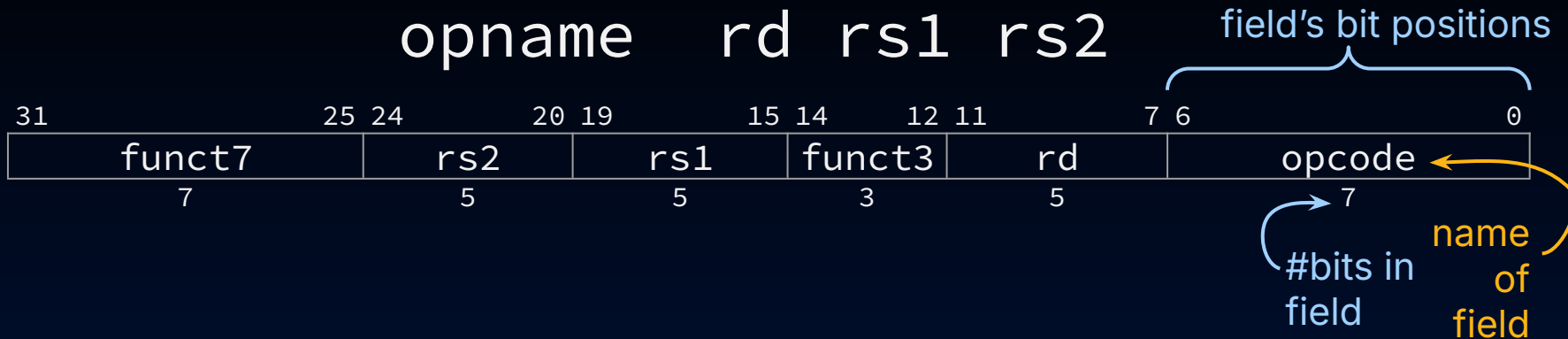


R-Type Instructions

- Machine Language
- R-Type Instructions
- I-Type Instructions
- I*-Type Instructions, Load Instructions
- S-Type Instructions (Store)



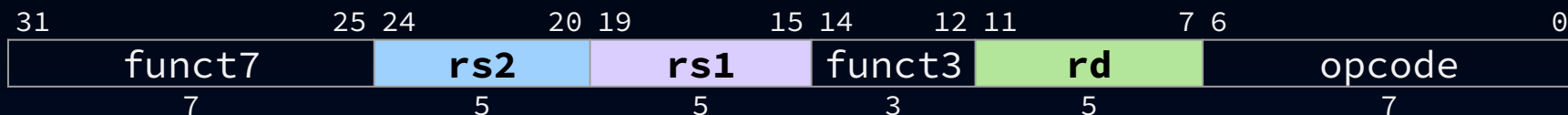
R-Type Instruction Format (1/3)



- Register-Register Arithmetic Instructions
 - All arithmetic functions that operate on registers (add, xor, sll, etc.) assume these bit fields.

R-Type Instruction Format (2/3)

opname rd rs1 rs2



"Source" Register 2
second operand

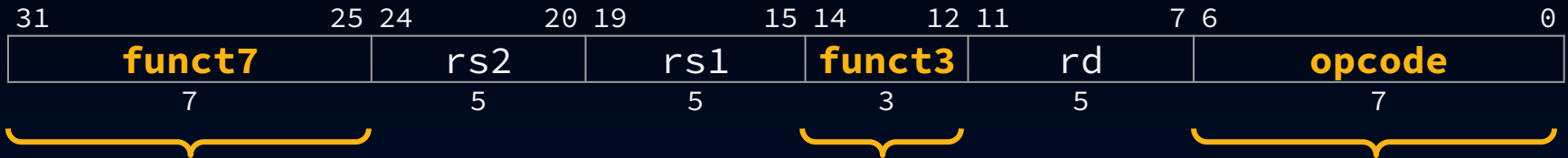
"Source" Register 1
second operand

"Destination" Register
gets result of
computation

- Register fields: rs1, rs2, rd
 - A 5-bit unsigned integer [0-31] corresponds to a register number (x0-x31).
 - $a0 \rightarrow x10 \rightarrow 0b01010$

R-Type Instruction Format (3/3)

opname rd rs1 rs2



All R-type instructions have the same 7-bit opcode.

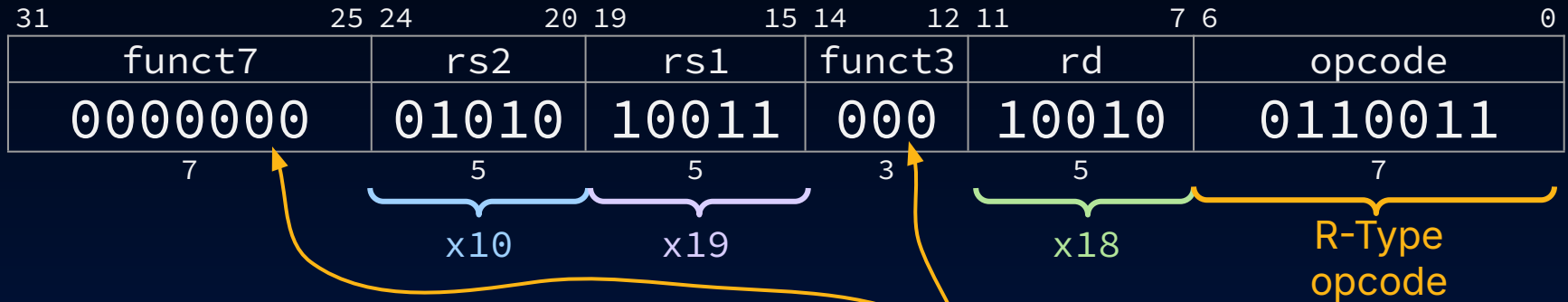
R-Type: opcode 0110011.

- Fields: funct7, funct3, opcode
 - What **instruction format** to use (opcode)
 - R-format: What **operation** to perform (funct3, funct7)
 - Why not just a 17-bit field for simplicity? (more later)



R-Type Example

add x18 x19 x10



1. Determine opcode, funct3, funct7 from reference card
2. Translate registers, etc.
3. (optional) Convert to hex





Your Turn: Hex → Instruction

Translate these 32 bits into a RISC-V instruction:

0x01B3 42B3

1. Convert to binary → find opcode → find instruction format type

0b0000 0001 1011 0011 0100 0010 1011 0011

- A. `slt x5 x6 x27`
- B. `slt x27 x5 x6`
- C. `slt x5 x27 x6`
- D. `xor s11 t0 t1`
- E. `xor t0 t1 s11`
- F. `xor t0 s11 t1`
- G. Something else



Yan, SP26



L12: Translate 0x01B3 42B3 to a RISC-V instruction. Binary: 0b0000 0001 1011 0011 0100 0010 1011 0011

A. slt x5 x6 x27

0%

B. slt x27 x5 x6

0%

C. slt x5 x27 x6

0%

D. xor s11 t0 t1

0%

E. xor t0 t1 s11

0%

F. xor t0 s11 t1

0%

G. Something else

0%





Your Turn: Hex → Instruction

E. xor t0 t1 s11

Translate these 32 bits into a RISC-V instruction:

0x01B3 42B3

1. Convert to binary → find opcode → find instruction format type

0b0000 0001 1011 0011 0100 0010 1011 0011

2. Split into fields according to instruction format (here, R-Type).
3. Determine opname with funct3, funct7.
4. Determine registers / register names.

31	25 24	20 19	15 14	12 11	7 6	0
funct7		rs2	rs1	funct3	rd	opcode
00000000		11011	00110	100	00101	0110011
		x27	x6		x5	R-format
		s11	t1		t0	

"xor"



RV32I R-Type (1/3): All 10 instructions

See reference card on [course website](#).

1. R-Type: How many unique opcode's?
2. R-Type: How many unique funct3's? unique funct7's?
3. Which instructions share funct3 fields?
 - a. What are their corresponding funct7 fields?
4. Why might it be *useful* to use "similar" field patterns?





RV32I R-Type (2/3)

See reference card on [course website](#).

1. R-Type: How many unique opcode's? One: 0110011
2. R-Type: How many unique funct3's? unique
funct7's? funct3: 8 unique
funct7: 2 unique
3. Which instructions share funct3 fields? add,sub
srl,sra
 - a. What are their corresponding funct7 fields?
4. Why might it be *useful* to use "similar" field patterns?

Easier to define CPU architecture! (Hint: bitmasks!)

	funct7			funct3		opcode	
2's comp negate rs2	0000000	rs2	rs1	000	rd	0110011	add
	0100000	rs2	rs1	000	rd	0110011	sub
	0000000	rs2	rs1	101	rd	0110011	srl
sign extend	0100000	rs2	rs1	101	rd	0110011	sra
	0000000	rs2	rs1	001	rd	0110011	sll

RV32I R-Type (3/3): Reference

Eight funct3 fields for ten instructions

	funct7			funct3		opcode	
2's comp negation of rs2	0000000	rs2	rs1	000	rd	0110011	add
	0100000	rs2	rs1	000	rd	0110011	sub
	0000000	rs2	rs1	111	rd	0110011	and
	0000000	rs2	rs1	110	rd	0110011	or
	0000000	rs2	rs1	100	rd	0110011	xor
sign extend	0000000	rs2	rs1	001	rd	0110011	sll
	0000000	rs2	rs1	101	rd	0110011	srl
	0100000	rs2	rs1	101	rd	0110011	sra
	0000000	rs2	rs1	010	rd	0110011	slt
	0000000	rs2	rs1	011	rd	0110011	sltu

set less than (see refcard)



I-Type Instructions

- Machine Language
- R-Type Instructions
- I-Type Instructions
- I*-Type Instructions, Load Instructions
- S-Type Instructions (Store)

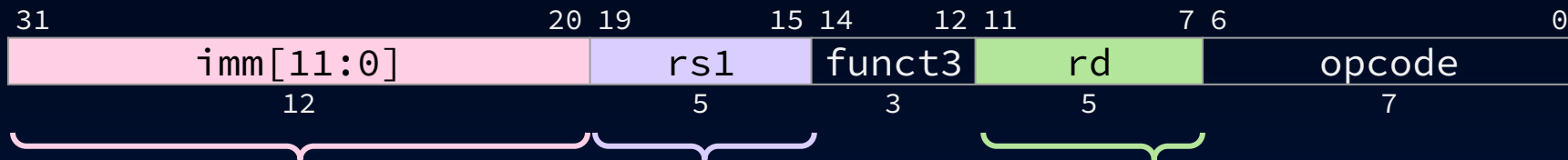


I-Type Instruction Format

- Register-Immediate Arithmetic Instructions
 - `addi`, `xori`, etc.
- ...and more

Immediates are integers **immediately** defined in the instruction itself.

opname rd rs1 imm



Imm[11:0] holds 12-bit wide immediate values

- Range: [-2048, +2047]
- CPU **sign-extends** to 32 bits before using in an arithmetic operation
- (more later) How to handle immediates > 12 bits?

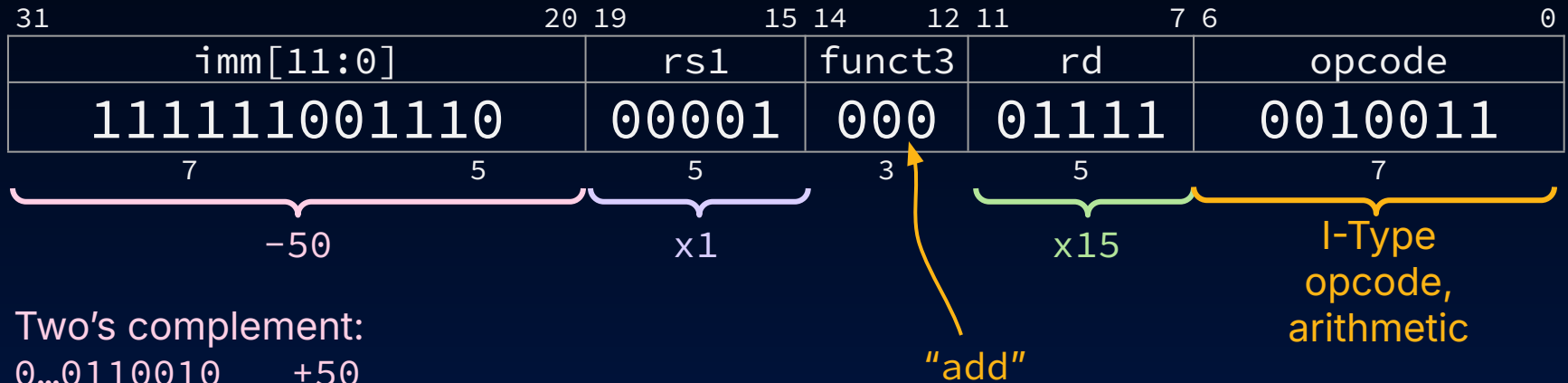
"Source" Register 1
second operand

"Destination" Register
gets result of computation



I-Type Example

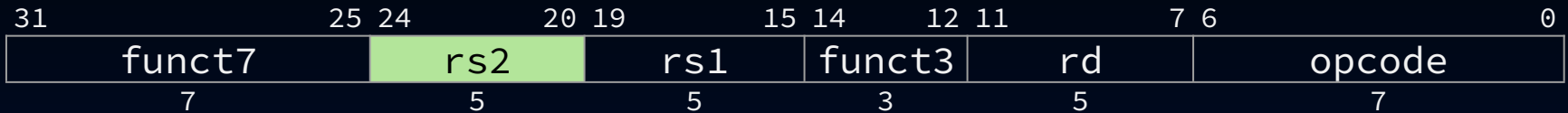
addi x15 x1 -50



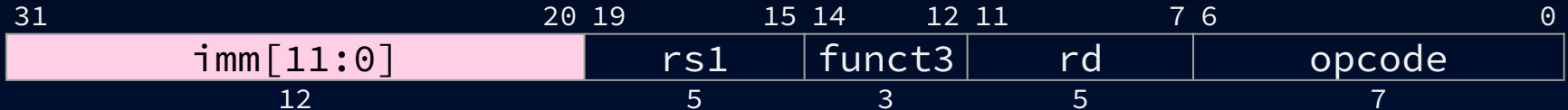


Immediate Fields Need to be Wider

R-Type: opname rd rs1 rs2



I-Type: opname rd rs1 imm



- If we used R-Type for immediates by replacing `rs2` with `imm`...
 - We'd only be able to represent 32 immediate values in a 5-bit field.
 - Immediates are often much bigger!
- Regardless, I-Type is mostly consistent with R-Type. Simplify how the CPU processes instructions!



Agenda

I*-Type Instructions, Load Instructions

- Machine Language
- R-Type Instructions
- I-Type Instructions
- I*-Type Instructions, Load Instructions
- S-Type Instructions (Store)



RV32I I-Type (1/6): All 17 Instructions

See reference card on [course website](#). **Consider both I-Type and I*-Type.**

1. I-Type: How many unique opcode's?
2. funct3: What similarities are there to R-type?
3. What are "I*-Type" instructions?
(What kind of arithmetic operation?)
 - a. → Why do we only need 5 bits for imm?
 - b. What are the upper 7 bits used for?
What similarities are there to R-type?
4. Why does load also use I-Type instruction format?
 - a. What about jalr?
 - b. (challenge) Why does store *not* use I-Type?



Yan, SP26





RV32I I-Type (2/6)

See reference card on [course website](#). Consider both I-Type and I*-Type.

1. I-Type: How many unique opcode's?

Arithmetic: 0010011

Load: 0000011

jalr: 1100111

Other: 1110011

2. funct3: What similarities are there to R-type?

- Same funct3 fields as corresponding R-format operation (remember, no subi)

funct3	
000	add
000	sub
111	and
110	or
100	xor
001	sll
101	srl
101	sra
010	slt
011	sltu

funct3	
000	addi
111	andi
110	ori
100	xori
001	slli
101	srli
101	srai
010	slti
011	sltiu



RV32I I-Type (3/6): "I*-Type"

3. What are "I*-Type" instructions?
(What kind of arithmetic operation?)

"I*-Type": "shift by immediate" instructions!

a. → Why do we only need 5 bits for `imm`?

`imm[4:0]`: max shift of 32 bits (anything larger will shift all data off register)

b. What are the upper 7 bits used for?
What similarities are there to R-type?

`imm[11:5]`: sign extend, similar to R-Type `funct7` field.

		funct3		opcode	
imm[11:0]		rs1	000	rd	0010011
0000000	imm[4:0]	rs1	001	rd	0110011
0000000	imm[4:0]	rs1	101	rd	0110011
0100000	imm[4:0]	rs1	101	rd	0110011

addi

slli

srli

srai

I-Type

"I*-Type"
(see 12-bit
imm field)

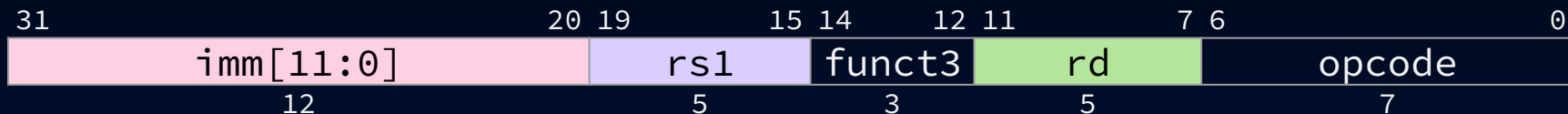
Yan, SP26

RV32I I-Type (4/6): Load Instructions

4. Why does load also use I-Type instruction format?

Need arithmetic immediate op:
compute memory address!

loadop rd imm(rs1)



Imm[11:0] "offset"

Add to base register to
form memory address

"Base" Register
Base address of
load

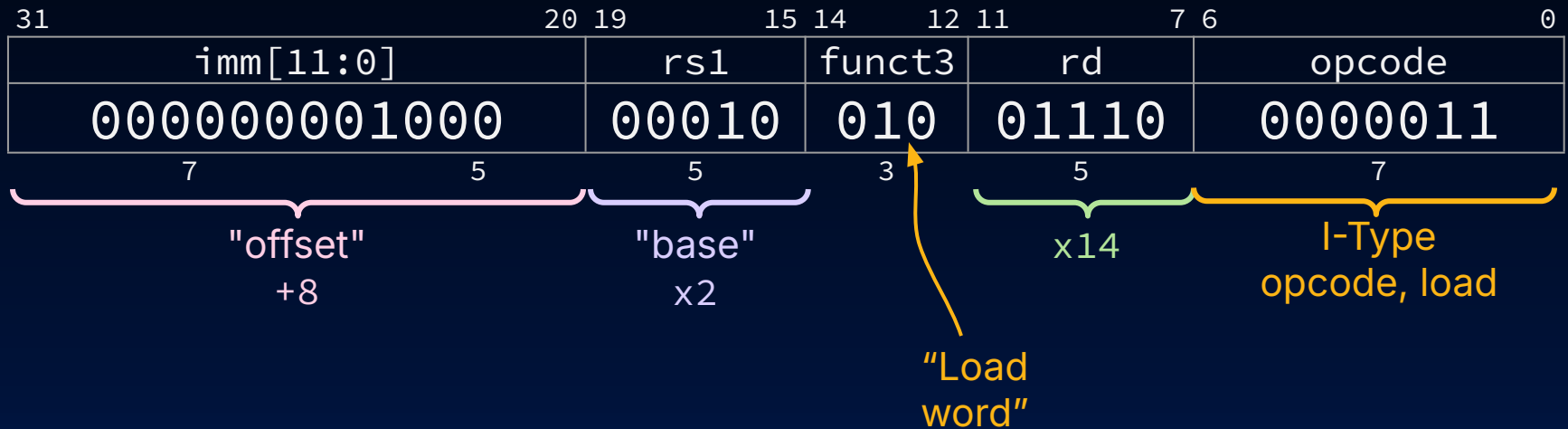
"Destination" Register
receives value loaded
from memory

Load address = (Base Register) + (Immediate Offset)



Load Example

lw x14 8(x2)



Load address = (Base Register) + (Immediate Offset)



Load Instructions

[review]

- `lb`: "load byte," `lh`: "load halfword" (16 bits)
 - **Sign extend** to fill upper bits of destination 32-bit register.
- `lbu`: "load unsigned byte," `lhu`: "load unsigned halfword"
 - **Zero extend** to fill upper bits of destination 32-bit register.
- Note no `lwu` instruction in RISC-V.
 - **Simplicity**: Nothing to extend when loading 32 bits to 32-bit register.
- (new) How is `funct3` designed? (Hint: data size, signedness)

funct3			opcode		
<code>imm[11:0]</code>	<code>rs1</code>	<code>000</code>	<code>rd</code>	<code>0000011</code>	<code>lb</code>
<code>imm[11:0]</code>	<code>rs1</code>	<code>100</code>	<code>rd</code>	<code>0000011</code>	<code>lbu</code>
<code>imm[11:0]</code>	<code>rs1</code>	<code>001</code>	<code>rd</code>	<code>0000011</code>	<code>lh</code>
<code>imm[11:0]</code>	<code>rs1</code>	<code>101</code>	<code>rd</code>	<code>0000011</code>	<code>lhu</code>
<code>imm[11:0]</code>	<code>rs1</code>	<code>010</code>	<code>rd</code>	<code>0000011</code>	<code>lw</code>

Yan, SP26



RV32I I-Type (5/6): Reference (Arithmetic)

		funct3		opcode	
imm[11:0]		rs1	000	rd	0010011 addi
imm[11:0]		rs1	111	rd	0010011 andi
imm[11:0]		rs1	110	rd	0010011 ori
imm[11:0]		rs1	100	rd	0010011 xori
0000000	imm[4:0]	rs1	001	rd	0010011 slli
0000000	imm[4:0]	rs1	101	rd	0010011 srli
0100000	imm[4:0]	rs1	101	rd	0010011 srai
imm[11:0]		rs1	010	rd	0010011 slti
imm[11:0]		rs1	011	rd	0010011 sltiu



RV32I I-Type (5/6): Reference (Load, Jump)

		funct3		opcode	
imm[11:0]	rs1	000	rd	0000011	lb
imm[11:0]	rs1	100	rd	0000011	lbu
imm[11:0]	rs1	001	rd	0000011	lh
imm[11:0]	rs1	101	rd	0000011	lhu
imm[11:0]	rs1	010	rd	0000011	lw
imm[11:0]	rs1	000	rd	1100111	jalr

ecall, ebreak out of scope



S-Format Layout (Store)

- Machine Language
- R-Type Instructions
- I-Type Instructions
- I*-Type Instructions, Load Instructions
- S-Type Instructions (Store)

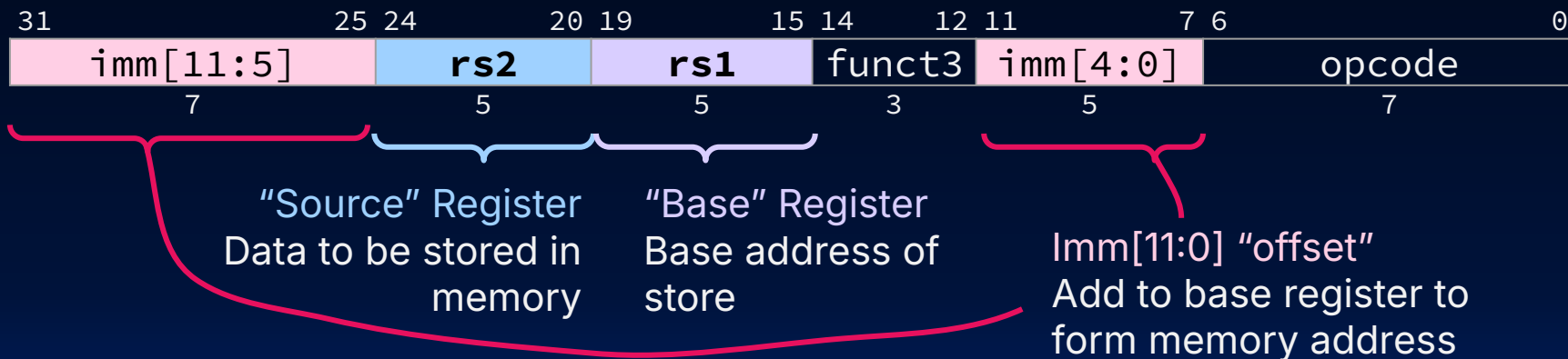


S-Type Instruction Format

- Store Instructions have their own format.
 - `sb`, `sh`, `sw`

Immediate's higher 7 bits and lower 5 bits are in separate fields!

`storeop rs2 imm(rs1)`

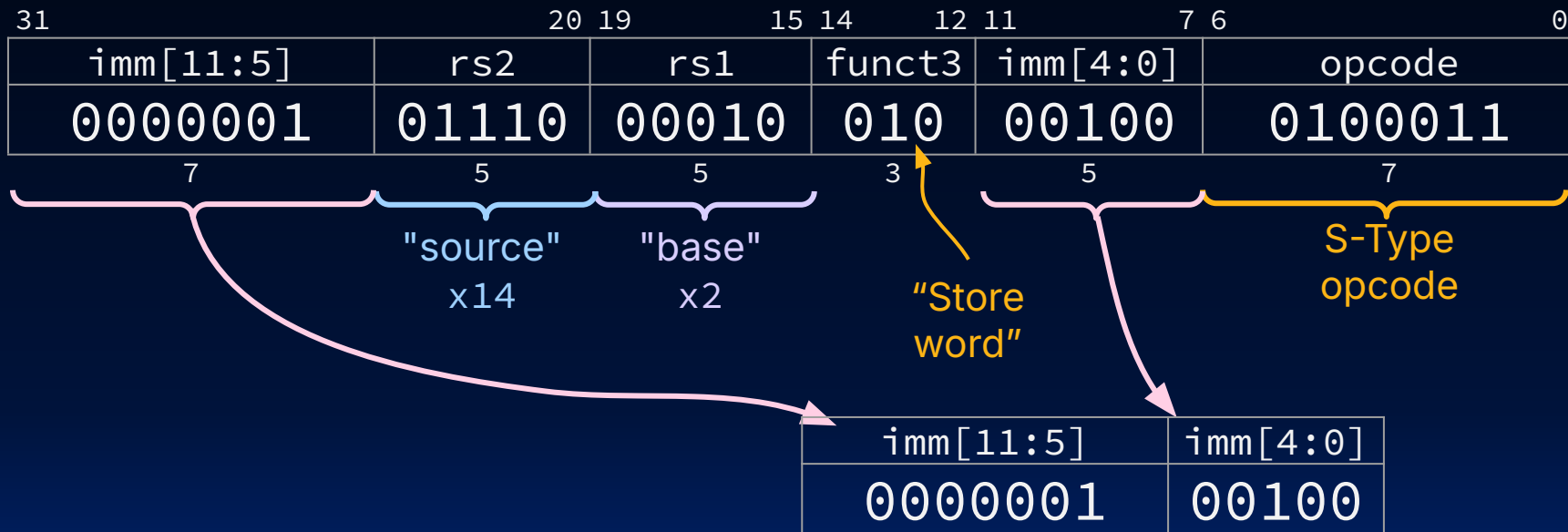


Store address = (Base Register) + (Immediate Offset)



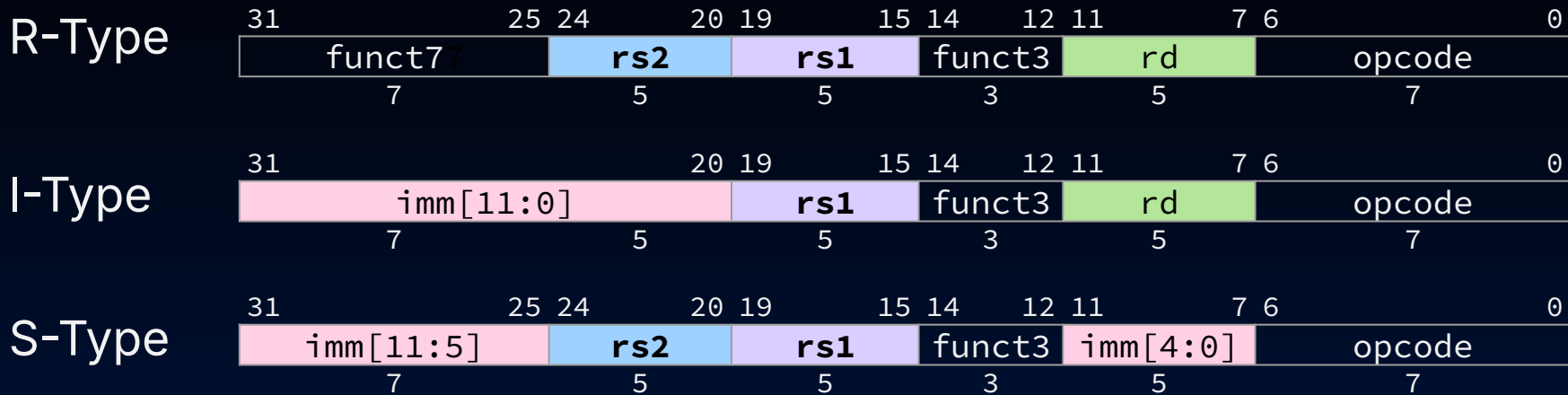
Store Example

sw x14 36(x2)





S-Type Simplifies Hardware Design



Why split up the immediate field in S-Type?

- RISC-V ISA design prioritizes keeping **register fields** in the same places:
 - "Read" registers in same bitrange across format types
 - "Write" registers, too
 - Immediates less critical

Critical for simple hardware design! (more later)



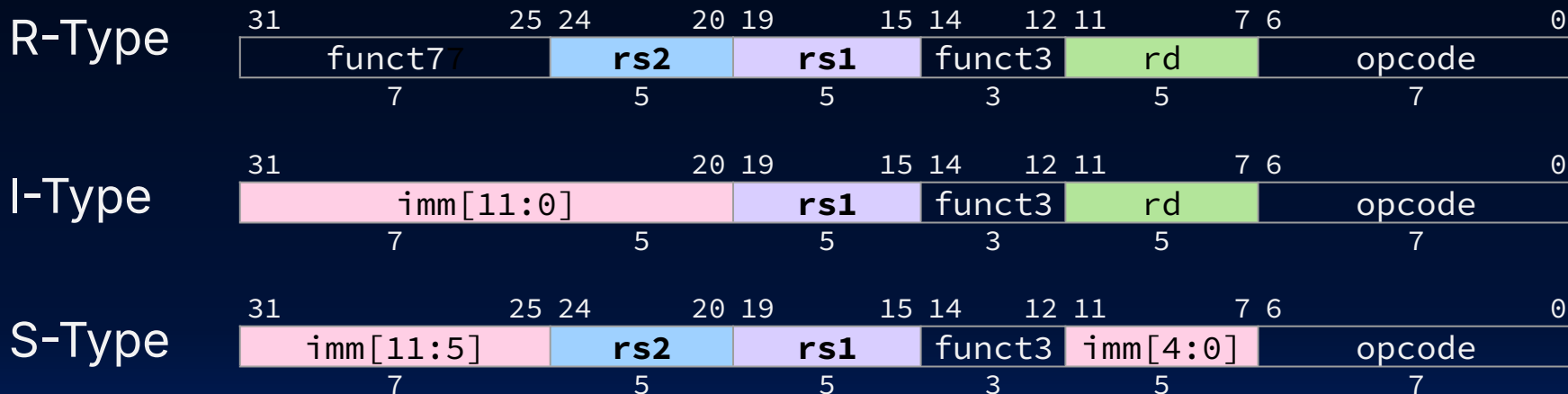
S-Type Instructions: Reference

- Recall store writes to memory only the specified bytes, so no sign/logical extension
- Same funct3 fields as load instructions

funct3				opcode		
imm[11:5]	rs2	rs1	000	imm[4:0]	0100011	sb
imm[11:5]	rs2	rs1	001	imm[4:0]	0100011	sh
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	sw

And in Conclusion...

- Simplification works for RISC-V: Instructions are same size as data word (32 bits) so that they use the same memory.
 - Computer program stored as a series of these 32-bit numbers.
- RISC-V **machine language instruction formats**:



- Next time: B-Type, U-Type, J-Type



Extra Exercises

- Machine Language
- R-Type Instructions
- I-Type Instructions
- I*-Type Instructions, Load Instructions
- S-Type Instructions (Store)



Your Turn

How do we encode `add x4 x3 x2 ?`

Lookup table:

funct3

funct7			funct3		opcode	
0000000	rs2	rs1	000	rd	0110011	add
0100000	rs2	rs1	000	rd	0110011	sub
0000000	rs2	rs1	100	rd	0110011	xor
0000000	rs2	rs1	111	rd	0110011	and

- A. 4021 8233_{hex}
- B. 0021 82b3_{hex}
- C. 4021 82b3_{hex}
- D. 0021 8233_{hex}
- E. 0021 8234_{hex}
- F. Something else

31	25	24	20	19	15	14	12	11	7	6	0	
funct7			rs2		rs1		funct3		rd		opcode	



Your Turn: Solution

How do we encode `add x4 x3 x2` ?

Lookup table:

funct3

funct7			funct3		opcode	
0000000	rs2	rs1	000	rd	0110011	add
0100000	rs2	rs1	000	rd	0110011	sub
0000000	rs2	rs1	100	rd	0110011	xor
0000000	rs2	rs1	111	rd	0110011	and

- A. 4021 8233_{hex}
- B. 0021 82b3_{hex}
- C. 4021 82b3_{hex}
- D. 0021 8233_{hex}**
- E. 0021 8234_{hex}
- F. Something else

31	25	24	20	19	15	14	12	11	7	6	0
funct7				rs2		rs1		funct3	rd		opcode
0000000				00010		00011		000	00100		0110011

hex

0

0

2

1

8

2

3

3